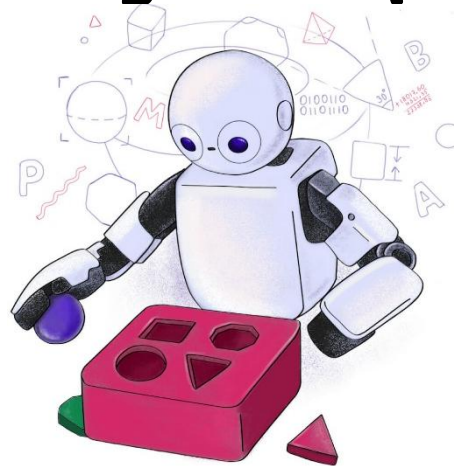


T320 - Introdução ao Aprendizado de Máquina II: *Classificação (Parte III)*



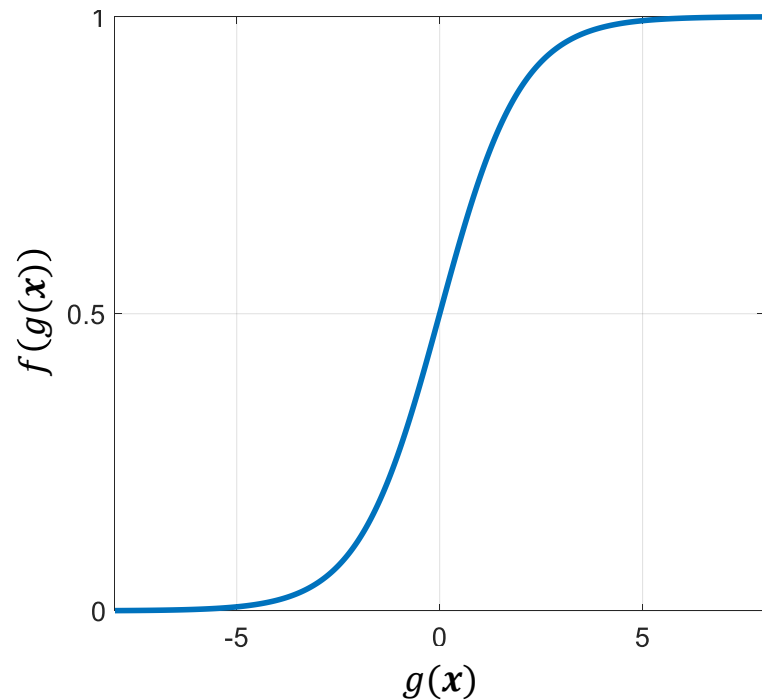
Inatel

Felipe Augusto Pereira de Figueiredo
felipe.figueiredo@inatel.br

Recapitulando

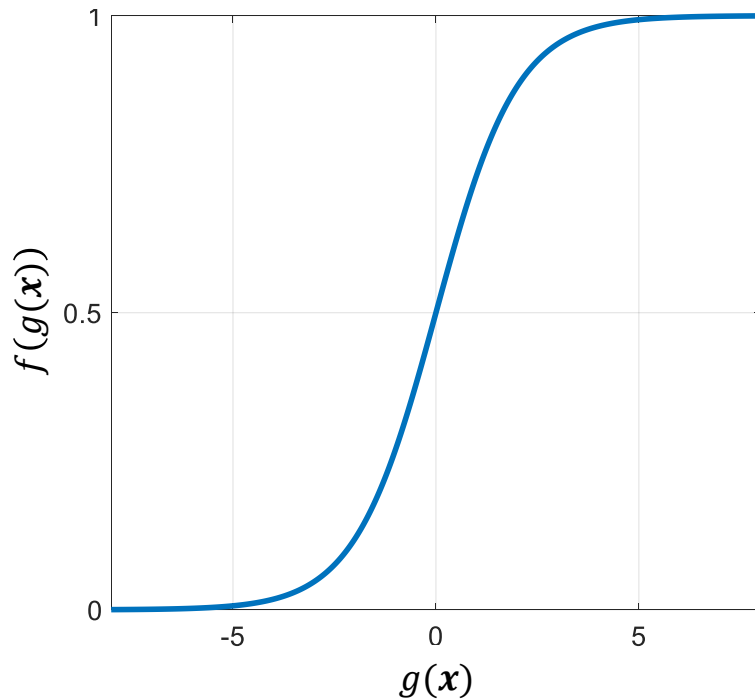
- Anteriormente, vimos que a função de ***limiar rígido*** apresenta alguns ***problemas*** como não poder ser utilizada para encontrar uma ***solução em forma fechada*** ou com o ***gradiente descendente***, além não nos dar a ***confiança das predições***.
- Aprendemos também, uma forma intuitiva e iterativa de encontrar os pesos da ***função discriminante*** quando usamos o ***limiar rígido***.
- Na sequência, usaremos ***outra função de limiar de decisão***, chamada de ***função logística***, com a qual é possível encontrar-se uma solução usando o ***gradiente descendente*** e termos o ***grau de confiança*** de uma predição.

Classificação com função de limiar logístico



- Em muitas situações, nós precisamos que o modelo indique o **grau de incerteza** quanto às predições.
- Todos esses problemas são resolvidos usando-se a função **sigmoide** (ou **logística**).
- Ela é **contínua**, **diferenciável** e **assume valores reais dentro do intervalo de 0 a 1**.

Classificação com função de limiar logístico



- A **função logística** é definida como

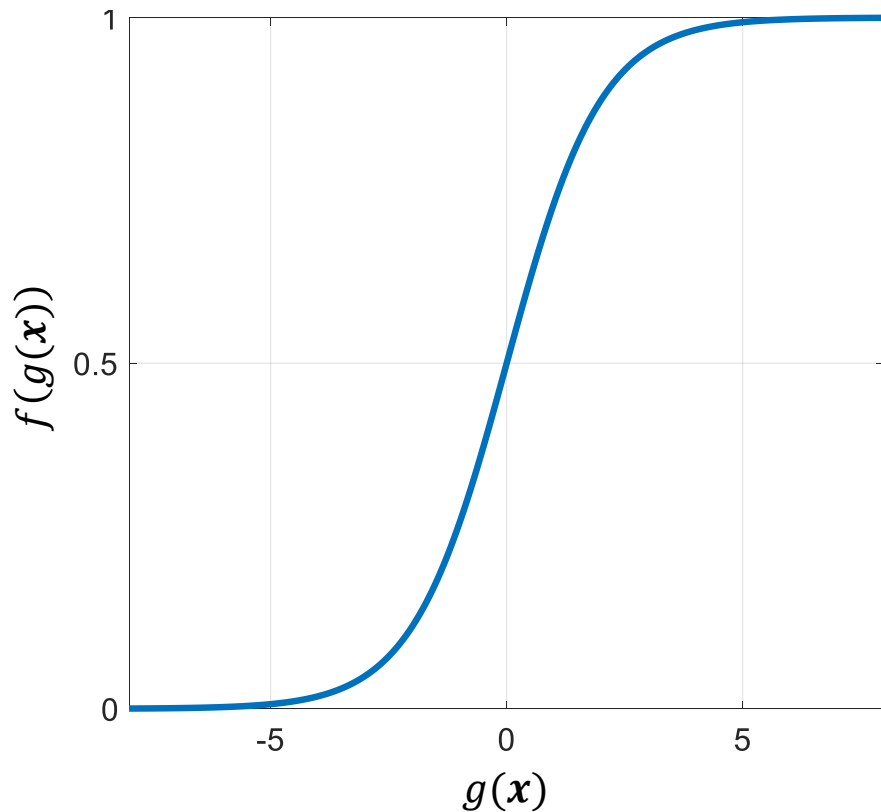
$$f(z) = \frac{1}{1+e^{-z}} \in [0, 1].$$

- Utilizando a **função logística** como **função de limiar de decisão**, temos a seguinte **função hipótese**

$$h_a(\mathbf{x}) = f(g(\mathbf{x})) = \frac{1}{1+e^{-g(\mathbf{x})}} \in [0, 1].$$

- Os valores de $h_a(\mathbf{x})$, ficam restritos ao intervalo $0 \leq h_a(\mathbf{x}) \leq 1$.
- **OBS.:** $g(\mathbf{x})$ pode assumir o formato de um **hiperplano** ou de um **polinômio**.

Classificação com função de limiar logístico



- A saída da **função hipótese**, $h_a(\mathbf{x})$ pode ser interpretada como a **probabilidade condicional da classe positiva**, C_2 , dado os vetores de atributos, $\mathbf{x}$, e o de pesos, $\mathbf{a}$.

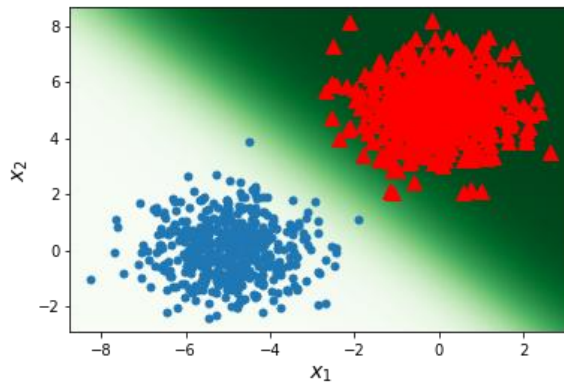
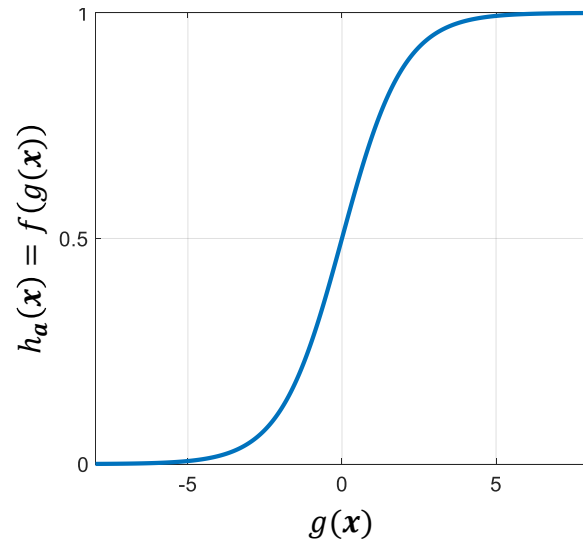
$$h_a(\mathbf{x}) = P(C_2 \mid \mathbf{x}; \mathbf{a}).$$

- Consequentemente, o complemento de $h_a(\mathbf{x})$, i.e.,

$$(1 - h_a(\mathbf{x})) = P(C_1 \mid \mathbf{x}; \mathbf{a}),$$

dá a **probabilidade condicional da classe negativa**, C_1 , dado os vetores de atributos, $\mathbf{x}$, e o de pesos, $\mathbf{a}$.

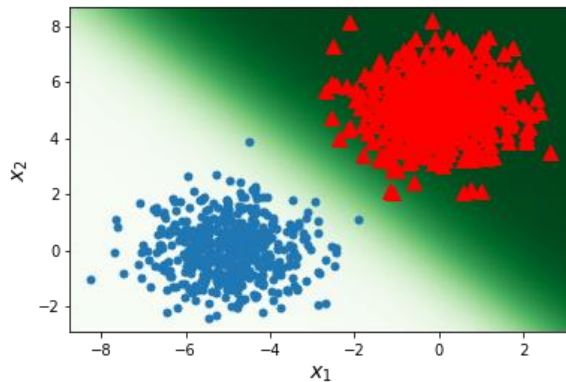
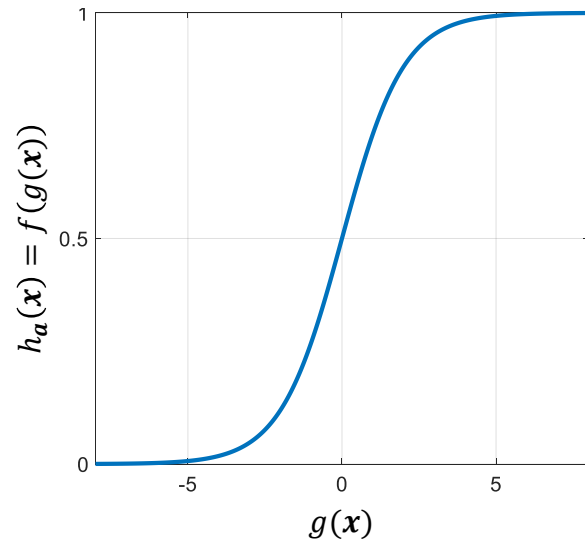
Classificação com função de limiar logístico



Fronteira de decisão suave.

- Dado que função logística tem uma **transição suave**, a nova **função hipótese**, $h_a(x)$, apresenta uma **transição suave entre as classes**.
- Esse comportamento é diferente do obtido com o **limiar rígido**, onde a **mudança entre classes é abrupta**.

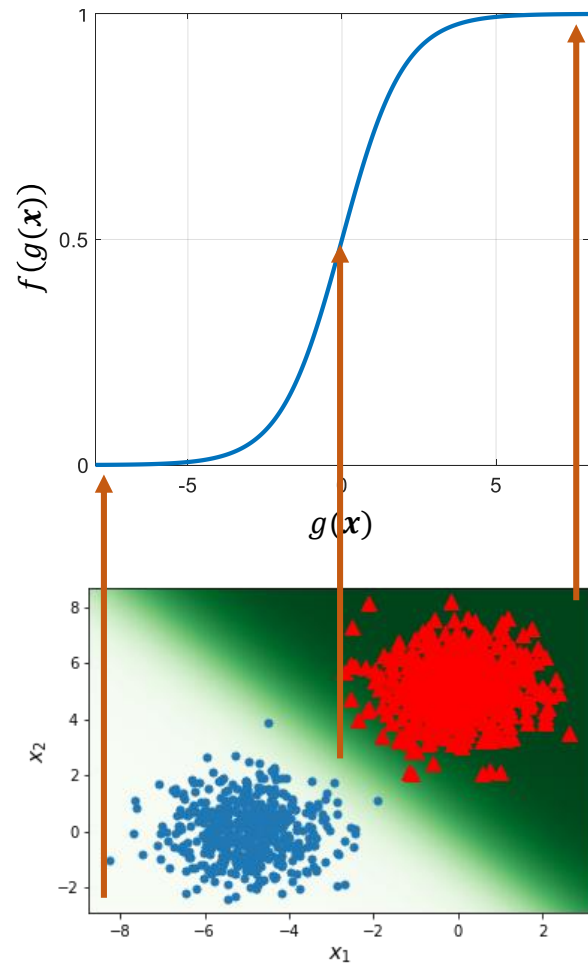
Classificação com função de limiar logístico



Fronteira de decisão suave.

- Lembrem-se que a **fronteira de decisão** é definida pela **função discriminante** e passa pelos pontos onde $g(x) = 0$.
- Assim, quanto **mais próximo** x estiver da **fronteira de decisão, mais próximo** o valor de $h_a(x)$ estará de **0.5**.
 - Se x estiver posicionado exatamente em cima da fronteira de decisão, $h_a(x) = 0.5$.
- Valores próximos a 0.5 indicam **indecisão** sobre a classe do exemplo.

Classificação com função de limiar logístico



Fronteira de decisão suave.

- Quanto **mais longe** x estiver da **fronteira de decisão**, **maior** será o **valor absoluto de** $g(x)$.
- Assim, quanto **mais longe** x estiver, **mais próximo** $h_a(x)$ **estará do valor 0 ou de 1** e, portanto, mais **certeza teremos sobre uma previsão**.

Regressão logística

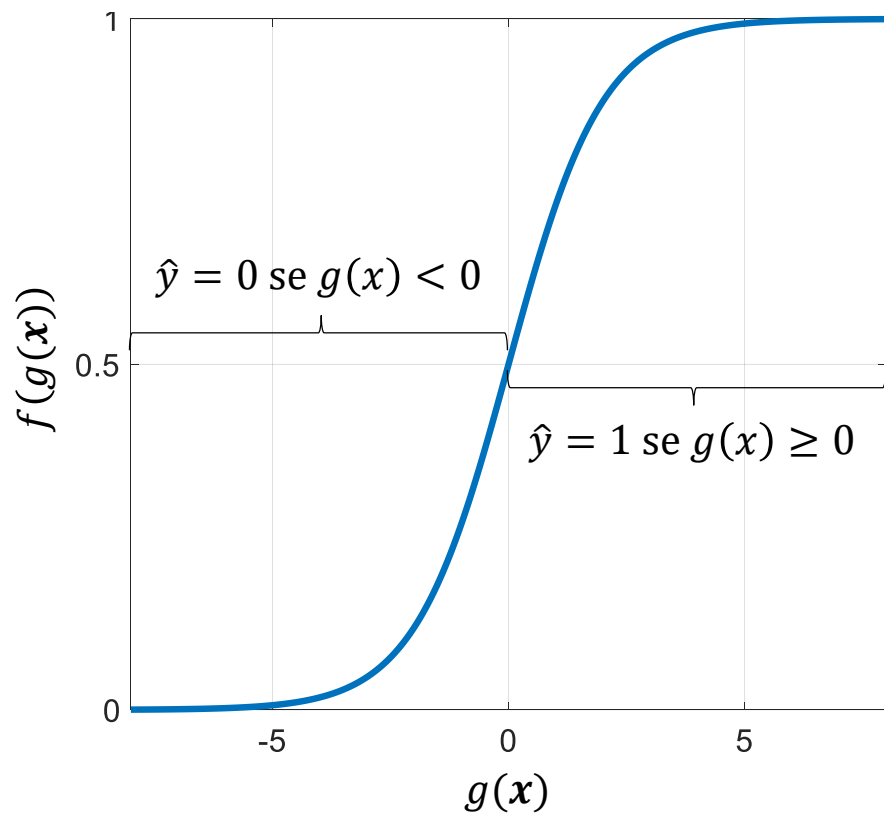
- Esse modelo que estima a **probabilidade** de um dado exemplo de entrada pertencer à classe positiva, **não é um classificador no sentido estrito**.
- Ele é na verdade um **regressor** e é chamado de **regressor logístico**.
- É um regressor, pois sua saída pode **assumir infinitos valores** no intervalo entre 0 e 1.
- O **regressor logístico** é normalmente usado para **classificação binária** (i.e., classificação entre duas classes, C_1 e C_2), mas, para isso, precisamos **quantizar a sua saída**.
- Toda a teoria por trás da **regressão logística** nos ajudará a entender o funcionamento das **redes neurais**.

Regressão logística

- Se quantiza a saída da **função hipótese**, $h_a(\mathbf{x})$, nos valores 0 ou 1 estabelecendo-se um **limiar de decisão**, T .
- Em geral, o **limiar de decisão** é feito igual a $T = 0.5$ (i.e., 50% de probabilidade).
- A saída **quantizada** do **regressor logístico** é dada por

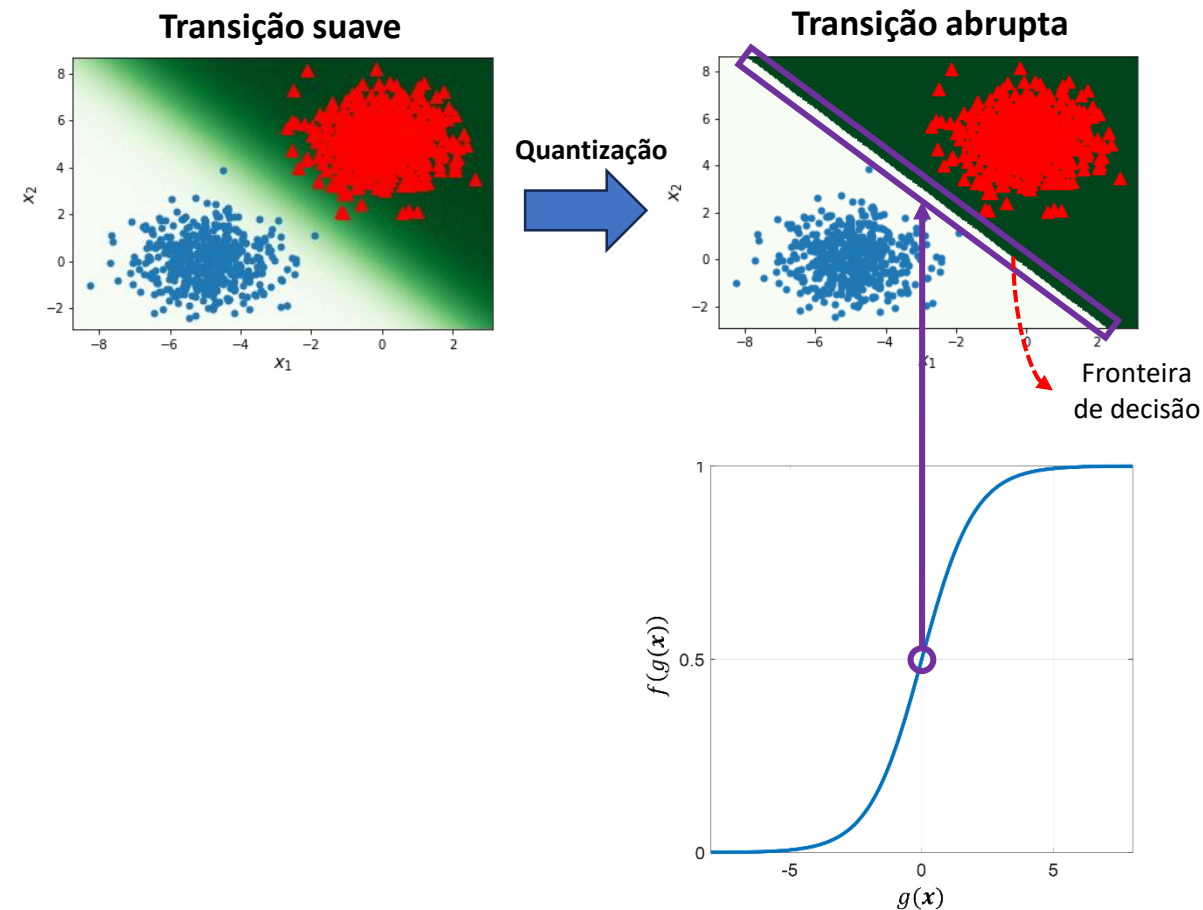
$$Classe = \hat{y} = \begin{cases} 0 \text{ (classe } C_1 \text{ – Negativa), se } h_a(\mathbf{x}) < 0.5 \\ 1 \text{ (classe } C_2 \text{ – Positiva), se } h_a(\mathbf{x}) \geq 0.5 \end{cases}$$

Regressão logística



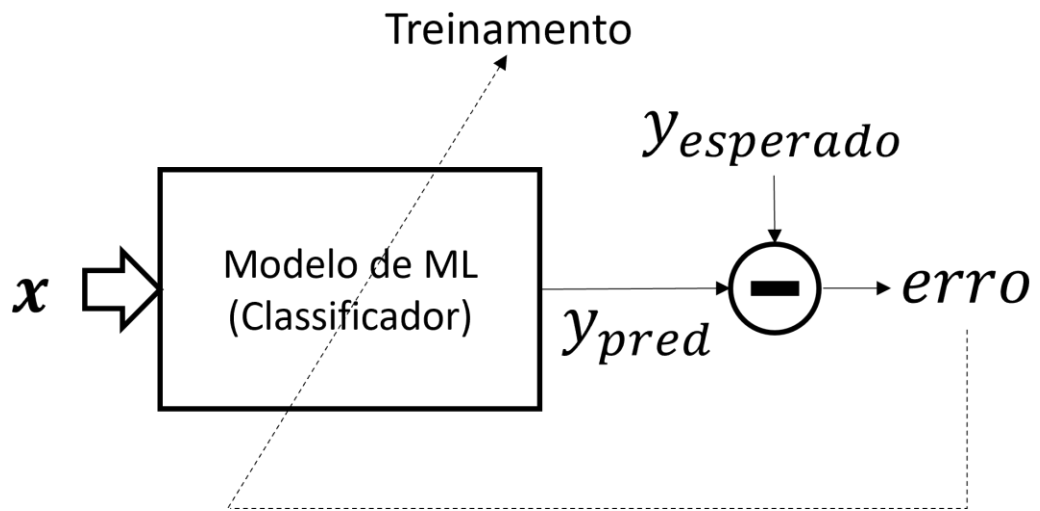
- Notem que
 - $f(g(x)) < 0.5$ quando $g(x) < 0$ e
 - $f(g(x)) \geq 0.5$ quando $g(x) \geq 0$.
- Portanto, o ***regressor logístico com limiar de quantização igual a 0.5*** prediz
 - a classe positiva, C_2 (i.e., $\hat{y} = 1$), se $g(x) \geq 0$ e
 - a classe negativa, C_1 (i.e., $\hat{y} = 0$), se $g(x) < 0$.

Propriedades da regressão logística



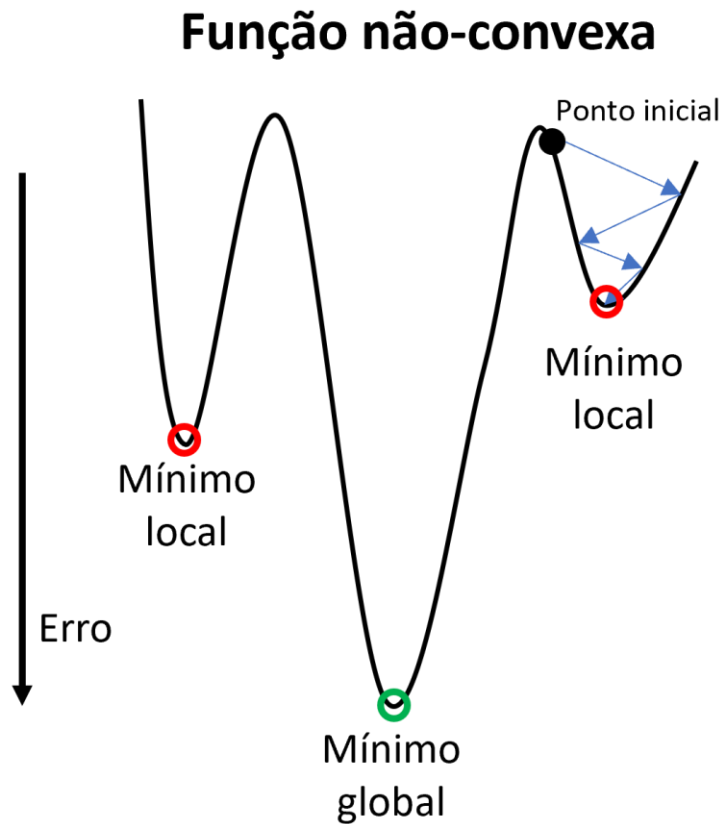
- A **transição entre classes** com o regressor logístico **é suave**, mas **após a quantização** de sua saída, ela **se torna abrupta**.
- A **fronteira de decisão** é determinada quando há uma **indecisão** entre as classes.
- Ou seja, quando
$$h_a(\mathbf{x}) = P(C_2 \mid \mathbf{x}; \mathbf{a}) = P(C_1 \mid \mathbf{x}; \mathbf{a}) = 0.5.$$

Função de erro



- Para **treinarmos** um **regressor logístico** e encontrarmos os **pesos** da **função discriminante**, nós **precisamos**, assim como fizemos com a **regressão linear**, **definir uma função de erro a ser minimizada**.
- Porém, adotar a função do **erro quadrático médio** como **função de erro não é uma boa escolha** para a **atualização dos pesos** no caso da **regressão logística** e **classificadores em geral** como veremos a seguir.

Função de erro

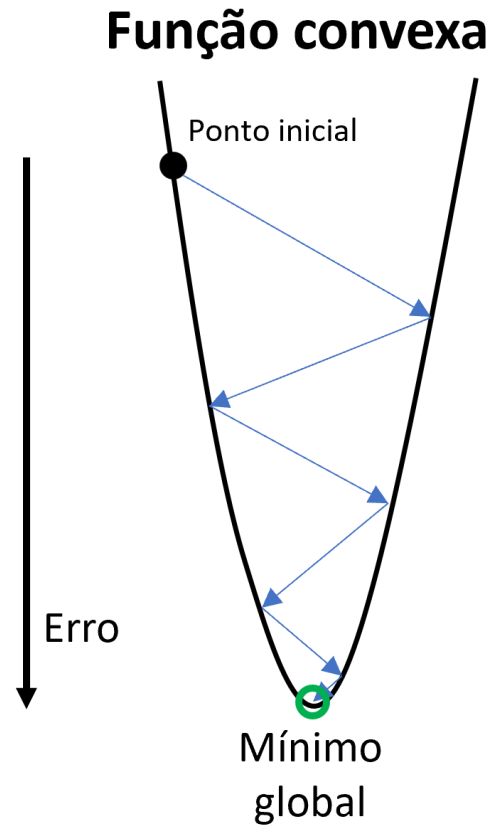


- Se considerarmos a **função de erro**, $J_e(\mathbf{a})$, como sendo o **erro quadrático médio**, temos

$$J_e(\mathbf{a}) = \frac{1}{N} \sum_{i=1}^N \left(y(i) - \underbrace{h_{\mathbf{a}}(\mathbf{x})}_{f(g(\mathbf{x}))} \right)^2.$$

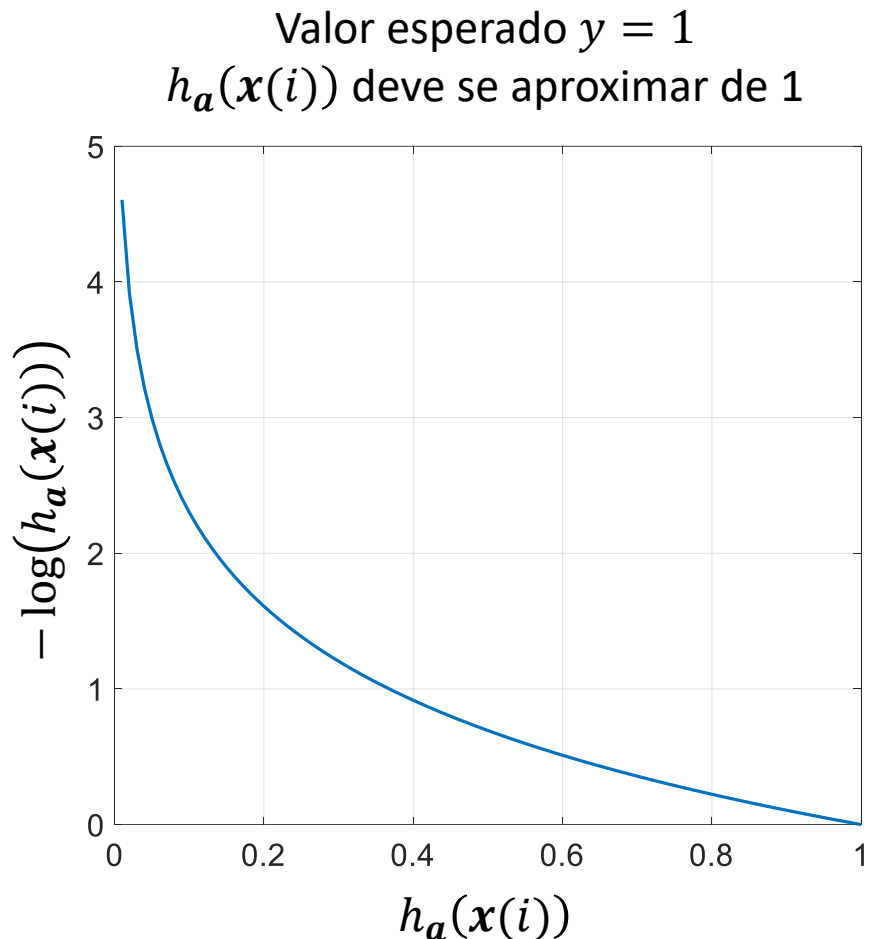
- Porém, como $f(\cdot)$ é uma **função não-linear**, $J_e(\mathbf{a})$ **não será**, consequentemente, **uma função convexa**, de forma que a **superfície de erro** poderá apresentar várias irregularidades, incluindo **diversos mínimos locais** que vão dificultar o aprendizado do modelo.
 - Por exemplo, o algoritmo do GD pode ficar preso em um mínimo local.

Função de erro



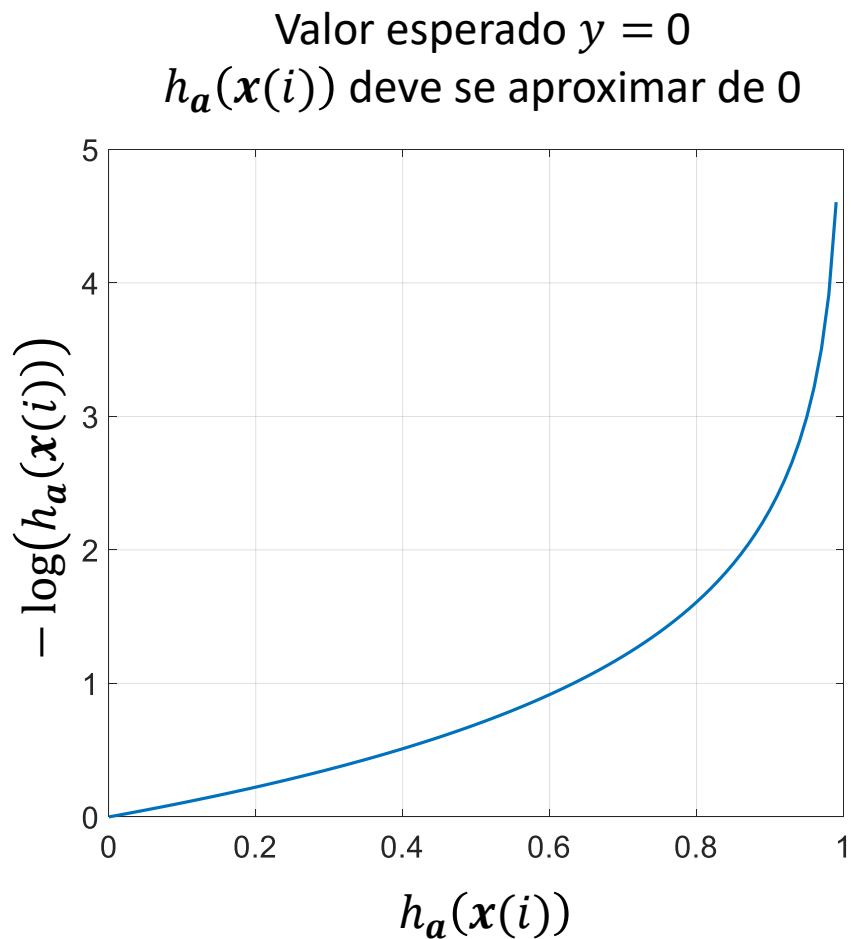
- **Ideia:** encontrar uma **função de erro** que tenha **superfície de erro** resultante **convexa**.
- Uma proposta **intuitiva** para a **função de erro para cada exemplo de entrada** é dada por
$$\text{Erro}(h_a(\mathbf{x}(i)); y(i)) = \begin{cases} -\log(h_a(\mathbf{x}(i))), & \text{se } y(i) = 1 \\ -\log(1 - h_a(\mathbf{x}(i))), & \text{se } y(i) = 0 \end{cases}$$
onde $y(i)$ é o i -ésimo rótulo.
- Veremos a seguir uma motivação para esta escolha.

Função de erro



- Quando o rótulo é $y = 1$, o valor de $-\log(h_a(\mathbf{x}(i)))$ se torna muito grande quando $h_a(\mathbf{x}(i))$ se aproxima de 0.
- Ou seja, o **erro será grande** se o regressor estimar uma probabilidade próxima a 0 para um exemplo da classe positiva, C_2 .
- Por outro lado, $-\log(h_a(\mathbf{x}(i)))$ se torna próximo de 0 quando $h_a(\mathbf{x}(i))$ se aproxima de 1.
- Portanto, o **erro será próximo de 0** se a probabilidade estimada for próxima de 1 para um exemplo da classe positiva, C_2 .

Função de erro



- Quando o rótulo é $y = 0$, i.e., exemplo da classe negativa, C_1 , o valor de $-\log(1 -$

Função de erro

- Nós podemos unir a **função de erro para cada exemplo** em uma expressão única **usando o valor dos rótulos**

$$\begin{aligned} & \text{Erro} \left(h_a(\mathbf{x}(i)); y(i) \right) \\ &= \underbrace{-y(i) \log \left(h_a(\mathbf{x}(i)) \right)}_{\text{Só exerce influência no erro se } y(i)=1} \underbrace{-(1 - y(i)) \log \left(1 - h_a(\mathbf{x}(i)) \right)}_{\text{Só exerce influência no erro se } y(i)=0}. \end{aligned}$$

- Assim, podemos definir a seguinte **função de erro médio**

$$J_e(\mathbf{a}) = -\frac{1}{N} \sum_{i=0}^{N-1} y(i) \log \left(h_a(\mathbf{x}(i)) \right) + (1 - y(i)) \log \left(1 - h_a(\mathbf{x}(i)) \right).$$

- Lembrem-se que sempre queremos **minimizar o erro ao longo de todo o conjunto de treinamento**, por isso tomamos a **média**.
- Essa função de erro é conhecida na literatura como **entropia cruzada**.

Função de erro

- A má notícia com relação a essa função é que **não existe uma equação de forma fechada** para encontrar os **pesos** que minimizem essa **função de erro**.
- A boa notícia é que essa **função de erro** é **convexa** e **diferenciável**.
- Consequentemente, **conseguimos** calcular o **vetor gradiente da função de erro** com relação aos pesos e **implementar** o algoritmo do **gradiente descendente (GD)**.
 - Podemos implementar todas as versões do **GD**.
- Por exemplo se usarmos o GDB, é garantido que ele encontre o **mínimo global** dado que a **taxa de aprendizagem** não seja muito grande e se espere tempo suficiente.

Processo de treinamento

- Da mesma forma como fizemos com a **regressão linear**, usamos o algoritmo do **gradiente descendente** para **encontrar os pesos que minimizam a função de erro médio**.
- A **equação de atualização iterativa** dos **pesos** é dada por

$$\mathbf{a} = \mathbf{a} - \alpha \frac{\partial J_e(\mathbf{a})}{\partial \mathbf{a}}.$$

- O **vetor gradiente** da **função de erro médio** é dado por

$$\frac{\partial J_e(\mathbf{a})}{\partial \mathbf{a}} = -\frac{1}{N} \sum_{i=0}^{N-1} [y(i) - h_{\mathbf{a}}(\mathbf{x}(i))] \mathbf{x}(i) = -\frac{1}{N} \mathbf{X}^T (\mathbf{y} - \hat{\mathbf{y}}).$$

Forma matricial

onde $\mathbf{X}$ é a matriz de atributos com dimensão $N \times K + 1$, $\mathbf{y}$ e $\hat{\mathbf{y}}$ são vetores coluna com dimensão $N \times 1$ contendo todos os valores esperados e preditos pelo regressor logístico, respectivamente.

Processo de treinamento

$$\frac{\partial J_e(\mathbf{a})}{\partial \mathbf{a}} = -\frac{1}{N} \sum_{i=0}^{N-1} [y(i) - h_a(\mathbf{x}(i))] \mathbf{x}(i) = -\frac{1}{N} \mathbf{X}^T (\mathbf{y} - \hat{\mathbf{y}})$$

- Para encontrar o **vetor gradiente**, a função $g(\mathbf{x})$ foi considerada como sendo a equação de um **hiperplano**, mas o resultado pode ser diretamente estendido para polinômios.
- Percebam que o **vetor gradiente** da **função de erro médio** para a **regressão logística** é **idêntico** (exceto pela constante 2) àquele obtido para a **regressão linear** utilizando a função de **erro quadrático médio**.
- **OBS.:** O **vetor gradiente** da **função de erro médio** vai variar dependendo da **função discriminante**, $g(\mathbf{x})$, adotada.
- Na sequência, veremos alguns exemplos com diferentes $g(\mathbf{x})$.

Vetor gradiente

- Se tivermos uma função discriminante com formato de **reta**

$$g(\mathbf{x}(i)) = a_0 + a_1 x_1(i) + a_2 x_2(i), \forall i.$$

- Como representamos essa função em formato matricial?

$$\mathbf{g}(\mathbf{X}) = \mathbf{X}\mathbf{a} \in \mathbb{R}^{N \times 1},$$

onde $\mathbf{X} = [\mathbf{x}_0, \mathbf{x}_1, \mathbf{x}_2] \in \mathbb{R}^{N \times K+1}$, $\mathbf{x}_0, \mathbf{x}_1$, e $\mathbf{x}_2 \in \mathbb{R}^{N \times 1}$ e $\mathbf{a} = [a_0, a_1, a_2]^T \in \mathbb{R}^{K+1 \times 1}$.

- Assim, basta usarmos a matriz de atributos, $\mathbf{X}$, acima para encontrar o vetor gradiente

$$\frac{\partial J_e(\mathbf{a})}{\partial \mathbf{a}} = -\frac{1}{N} \mathbf{X}^T (\mathbf{y} - \hat{\mathbf{y}}).$$

Vetor gradiente

- Se tivermos uma função discriminante com formato de um **círculo**

$$g(\mathbf{x}(i)) = a_0 + x_1^2(i) + x_2^2(i), \forall i.$$

- Como representamos essa função em formato matricial?

$$\mathbf{g}(\mathbf{X}) = \mathbf{X}\mathbf{a} \in \mathbb{R}^{N \times 1},$$

onde $\mathbf{X} = [\mathbf{x}_0, \mathbf{x}_1^2, \mathbf{x}_2^2] \in \mathbb{R}^{N \times K+1}$, $\mathbf{x}_0, \mathbf{x}_1^2$, e $\mathbf{x}_2^2 \in \mathbb{R}^{N \times 1}$ e $\mathbf{a} = [a_0, a_1, a_2]^T \in \mathbb{R}^{K+1 \times 1}$.

- Assim, basta usarmos a matriz de atributos, $\mathbf{X}$, acima para encontrar o vetor gradiente

$$\frac{\partial J_e(\mathbf{a})}{\partial \mathbf{a}} = -\frac{1}{N} \mathbf{X}^T (\mathbf{y} - \hat{\mathbf{y}}).$$

Vetor gradiente

- Se tivermos uma função discriminante com formato de um *hipérbole retangular*

$$g(\mathbf{x}(i)) = a_0 + a_1 x_1(i) * x_2(i), \forall i.$$

- Como representamos essa função em formato matricial?

$$\mathbf{g}(\mathbf{X}) = \mathbf{X}\mathbf{a} \in \mathbb{R}^{N \times 1},$$

onde $\mathbf{X} = [\mathbf{x}_0, \mathbf{x}_1 \odot \mathbf{x}_2] \in \mathbb{R}^{N \times K+1}$, $\mathbf{x}_0, \mathbf{x}_1, \mathbf{x}_2$, e $\mathbf{x}_1 \odot \mathbf{x}_2 \in \mathbb{R}^{N \times 1}$, $\mathbf{a} = [a_0, a_1]^T \in \mathbb{R}^{K+1 \times 1}$ e $\odot$ é a multiplicação elemento-a-elemento.

- Assim, basta usarmos a matriz de atributos, $\mathbf{X}$, acima para encontrar o vetor gradiente

$$\frac{\partial J_e(\mathbf{a})}{\partial \mathbf{a}} = -\frac{1}{N} \mathbf{X}^T (\mathbf{y} - \hat{\mathbf{y}}).$$

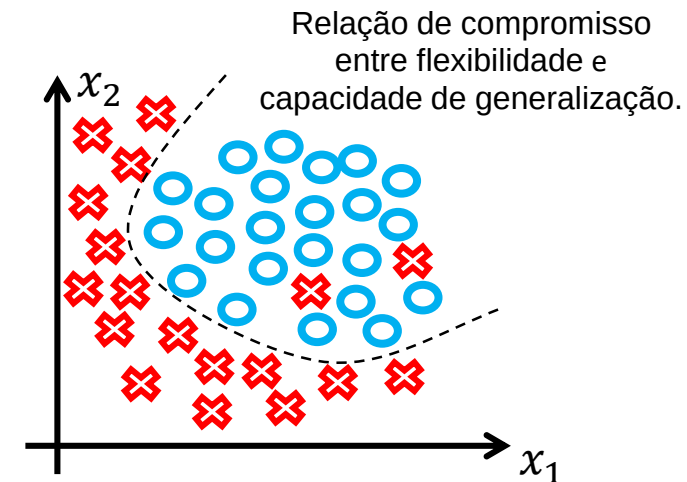
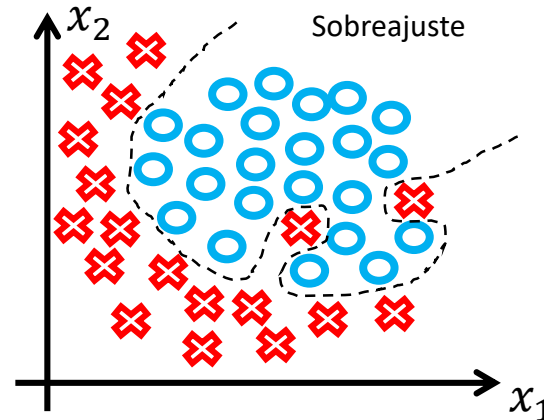
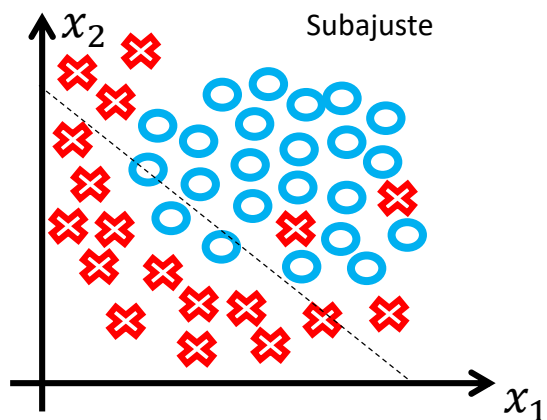
Vetor gradiente

- Agora, de posse do ***vetor gradiente***, podemos usá-lo com o ***gradiente descendente*** (nas versões em batelada, estocástico ou mini-batch) para atualizar os pesos.

$$\mathbf{a} = \mathbf{a} - \alpha \frac{\partial J_e(\mathbf{a})}{\partial \mathbf{a}}.$$

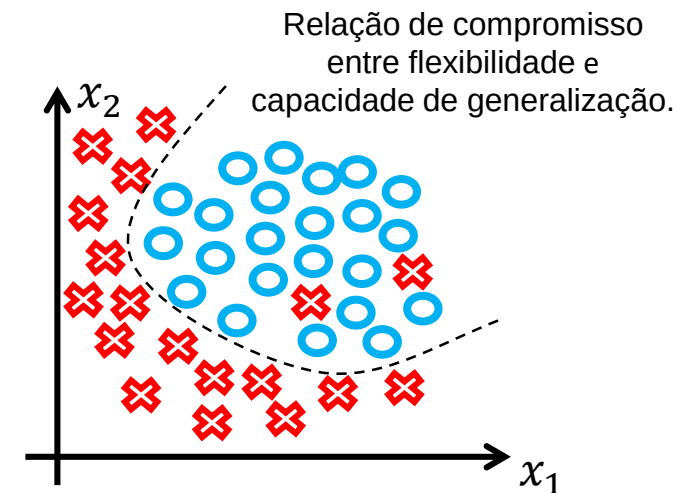
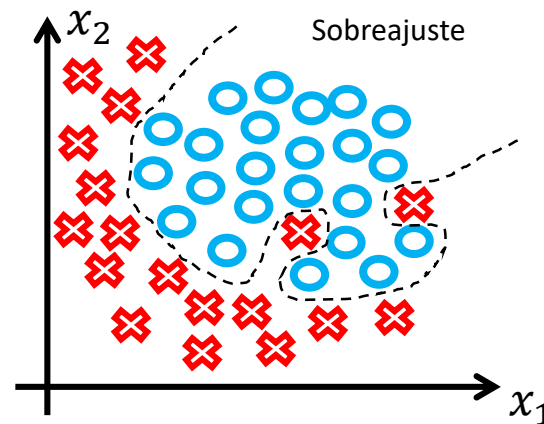
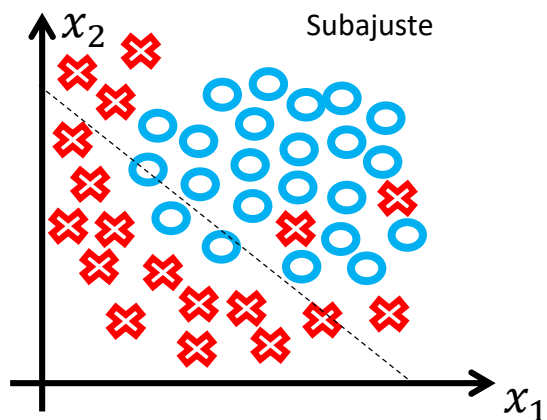
Subajuste e sobreajuste

- Como vimos, a **função discriminante**, $g(\mathbf{x})$, pode assumir também o formato de um **polinômio**.
- Porém, muitas vezes, nós não sabemos qual a **melhor ordem** para este polinômio.
- Assim, como ocorre no caso da **regressão linear**, modelos de **regressão logística** também estão sujeitos à ocorrência de **sobreajuste** e **subajuste**.



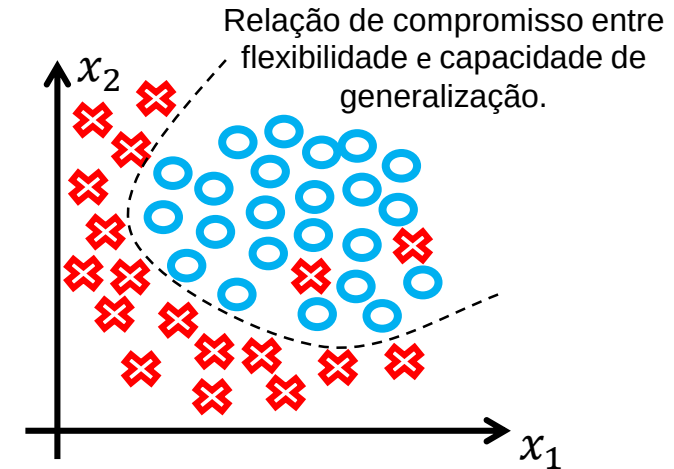
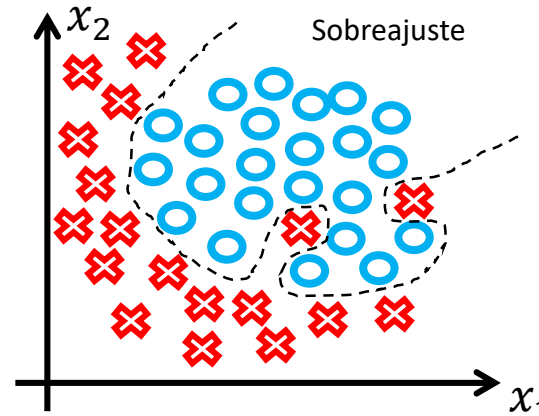
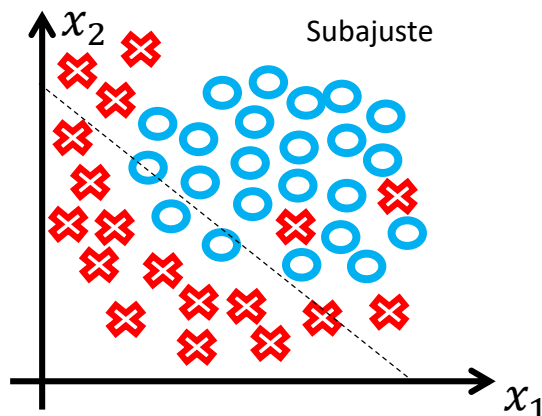
Subajuste e sobreajuste

- Na primeira figura, o polinômio (i.e., reta) tem uma **baixa flexibilidade**.
- O modelo também apresenta uma **baixa capacidade de generalização**.
- Assim, os erros nos conjuntos de treinamento e teste seriam **ambos altos**.
- Ou seja, o classificador erraria a classificação de boa parte dos exemplos de ambos os conjuntos.



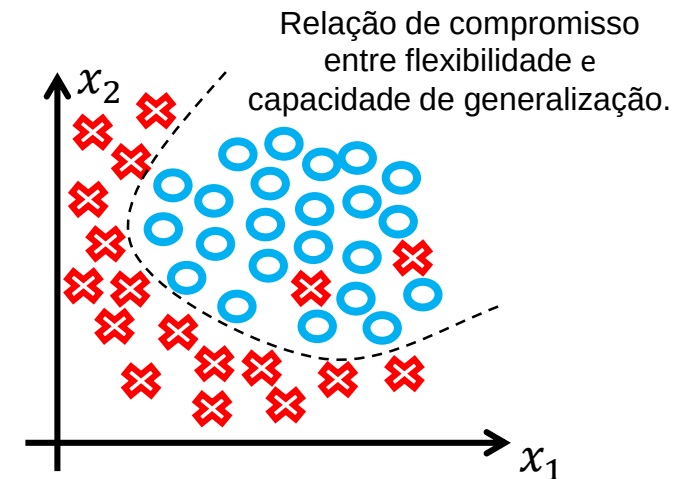
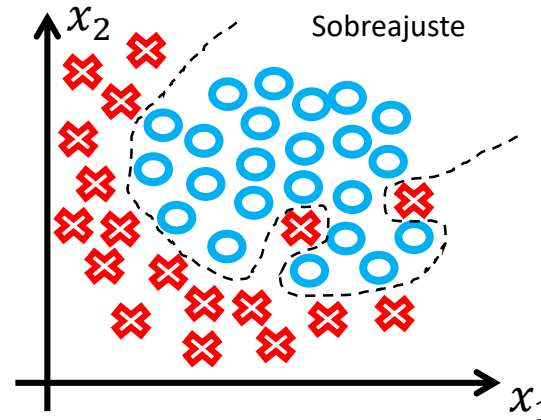
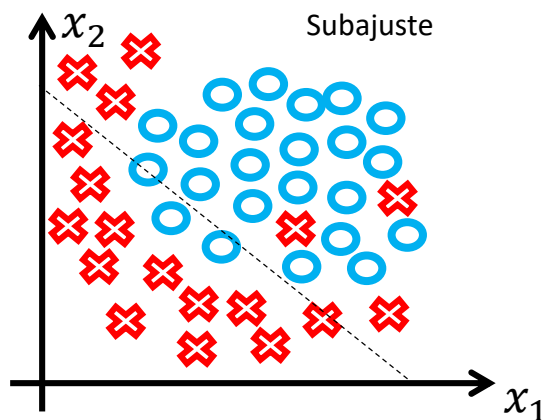
Subajuste e sobreajuste

- Na segunda figura, a **flexibilidade excessiva** do polinômio (i.e., ordem elevada) dá origem a contorções na **fronteira de decisão** na tentativa de **minimizar o erro** de classificação **junto aos dados de treinamento**.
- Porém, o classificador cometerá muitos erros para dados inéditos, i.e., **não irá generalizar bem**.
- Nesse caso o classificador apresenta **erro de treinamento muito baixo** e **erro de teste muito alto**.



Subajuste e sobreajuste

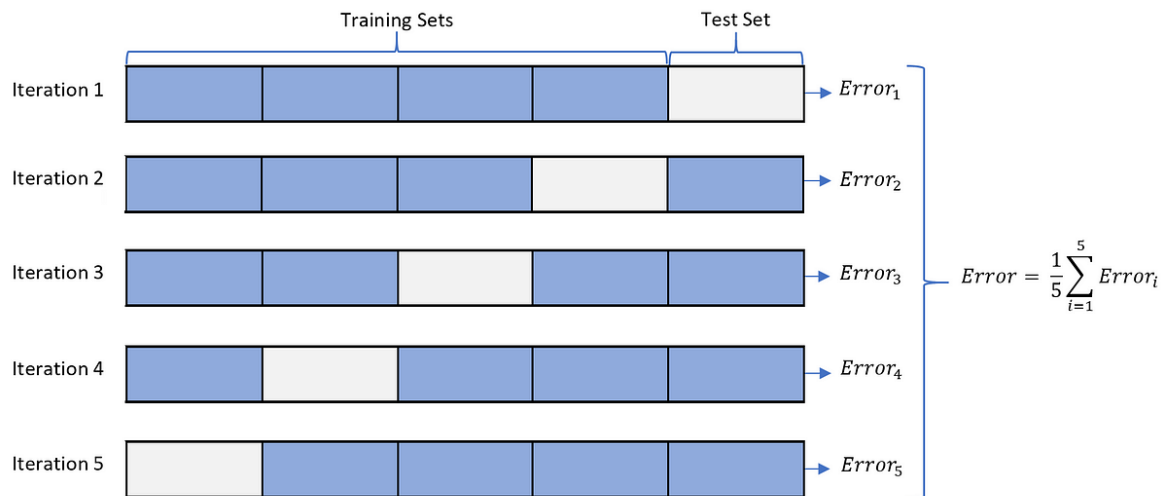
- Já a última figura mostra o que seria uma boa **hipótese de classificação**.
- O polinômio usado apresenta uma **boa relação de compromisso** entre a **flexibilidade** e a **capacidade de generalização**.
- Os erros nos conjuntos de treinamento e de teste seriam **baixos e próximos**.



Como evitamos o subajuste e
sobreajuste?

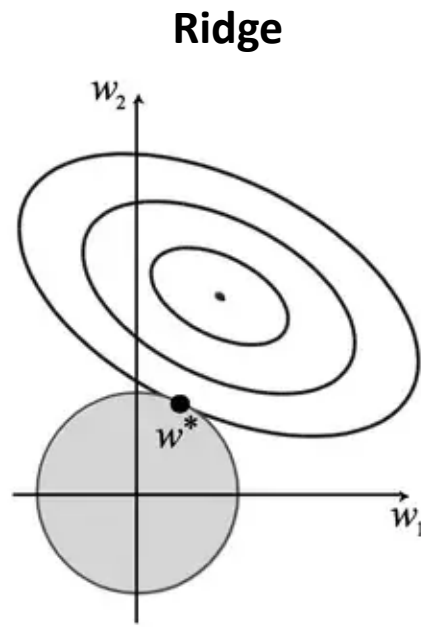
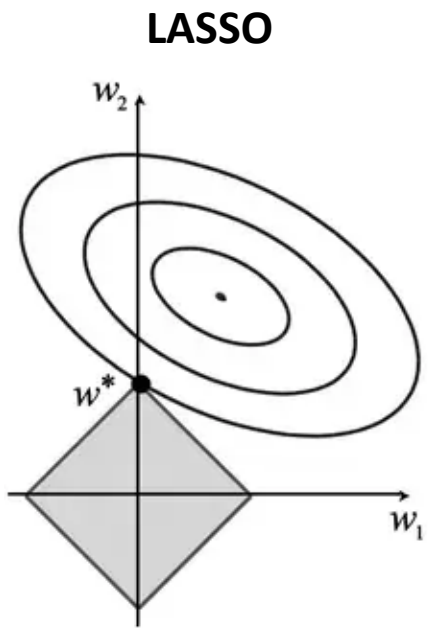
Como evitamos o subajuste e sobreajuste?

k-fold



- Quando não conhecemos a **melhor ordem para o polinômio** da **função discriminante**, $g(x)$, podemos usar **técnicas de validação cruzada** (e.g., *holdout* ou *k-fold*) para encontra-lo.
- Essas técnicas nos auxiliam a encontrar um polinômio que apresente uma **relação de compromisso entre flexibilidade e capacidade de generalização**, evitando assim os dois problemas.

Como evitamos o subajuste e sobreajuste?



- Uma forma de **evitar apenas problemas de sobreajuste** é usar **técnicas de regularização** (e.g., LASSO, Ridge, *Elastic-Net*, *Early-stopping*).
- A regularização **reduz a flexibilidade** do modelo por meio de **penalizações** aplicadas a seus pesos.
 - Modelos que se **sobreajustam** têm **pesos com valores absolutos muito altos**.
 - O que as técnicas de regularização fazem é **restringir o aumento** dos pesos a uma **região** de possíveis valores.

Tarefas

- **Quiz:** “*T320 - Quiz - Classificação (Parte III)*” que se encontra no MS Teams.
- **Exercício Prático:** [Laboratório #3](#).
 - Pode ser acessado através do link acima (Google Colab) ou no GitHub.
 - Se atentem aos prazos de entrega.
 - [Instruções para resolução e entrega dos laboratórios](#).

Obrigado!

Anexo I: Encontrando o vetor gradiente do regressor logístico

Encontrando o vetor gradiente

- Antes de encontrarmos o **vetor gradiente** de $J_e(\mathbf{a})$, vamos reescrever a **função de erro** utilizando as seguintes equivalências

$$\log(h_{\mathbf{a}}(\mathbf{x}(i))) = \log\left(\frac{1}{1+e^{-\mathbf{x}(i)^T \mathbf{a}}}\right) = -\log\left(1 + e^{-\mathbf{x}(i)^T \mathbf{a}}\right),$$

$$\log(1 - h_{\mathbf{a}}(\mathbf{x}(i))) = \log\left(1 - \frac{1}{1+e^{-\mathbf{x}(i)^T \mathbf{a}}}\right) = -\mathbf{x}(i)^T \mathbf{a} - \log\left(1 + e^{-\mathbf{x}(i)^T \mathbf{a}}\right).$$

- Assim, a nova expressão para a **função de erro médio** é dada por

$$\begin{aligned} J_e(\mathbf{a}) &= -\frac{1}{N} \sum_{i=0}^{N-1} -y(i) \log\left(1 + e^{-\mathbf{x}(i)^T \mathbf{a}}\right) + (1 \\ &\quad - y(i)) \left[-\mathbf{x}(i)^T \mathbf{a} - \log\left(1 + e^{-\mathbf{x}(i)^T \mathbf{a}}\right) \right] \end{aligned}$$

Encontrando o vetor gradiente

- O termo $-y(i) \log(1 + e^{-x(i)^T \mathbf{a}})$ é cancelado com um dos elementos gerados a partir do produto envolvido no segundo termo, de forma que

$$J_e(\mathbf{a}) = -\frac{1}{N} \sum_{i=0}^{N-1} -\mathbf{x}(i)^T \mathbf{a} + y(i) \mathbf{x}(i)^T \mathbf{a} - \log(1 + e^{-x(i)^T \mathbf{a}}).$$

- Se $-\mathbf{x}(i)^T \mathbf{a} = -\log(e^{x(i)^T \mathbf{a}})$, então

$$-\mathbf{x}(i)^T \mathbf{a} - \log(1 + e^{-x(i)^T \mathbf{a}}) = -\log(1 + e^{x(i)^T \mathbf{a}}).$$

- Desta forma, a **função de erro médio** se torna

$$J_e(\mathbf{a}) = -\frac{1}{N} \sum_{i=0}^{N-1} y(i) \mathbf{x}(i)^T \mathbf{a} - \log(1 + e^{x(i)^T \mathbf{a}}).$$

- Em seguida, encontramos o **vetor gradiente** de cada termo da equação acima.

Encontrando o vetor gradiente

- Assim, o **vetor gradiente** do primeiro termo da equação anterior é dado por

$$\frac{\partial [y(i)\mathbf{x}(i)^T \mathbf{a}]}{\partial \mathbf{a}} = y(i)\mathbf{x}(i)$$

- O **vetor gradiente** do segundo termo da equação anterior é dado por

$$\begin{aligned} \frac{\partial \left[\log \left(1 + e^{\mathbf{x}(i)^T \mathbf{a}} \right) \right]}{\partial \mathbf{a}} &= \frac{1}{1 + e^{\mathbf{x}(i)^T \mathbf{a}}} e^{\mathbf{x}(i)^T \mathbf{a}} \mathbf{x}(i) \\ &= \frac{1}{1 + e^{-\mathbf{x}(i)^T \mathbf{a}}} \mathbf{x}(i) \\ &= h_{\mathbf{a}}(\mathbf{x}(i)) \mathbf{x}(i). \end{aligned}$$


- Usamos a **regra da cadeia** para encontrar o vetor gradiente do segundo termo.

Encontrando o vetor gradiente

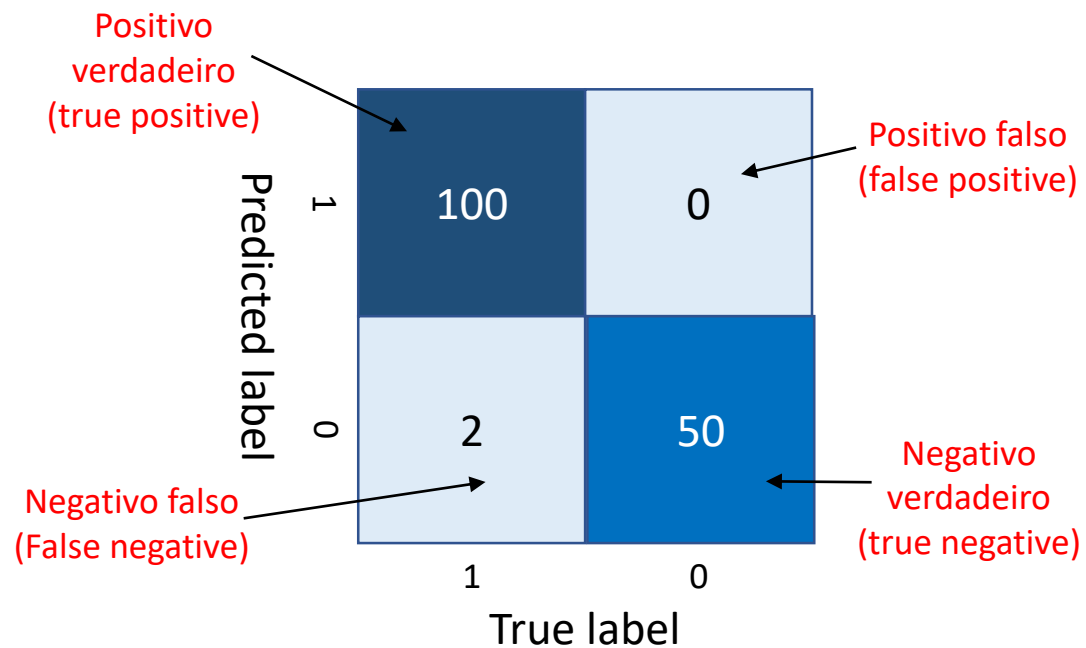
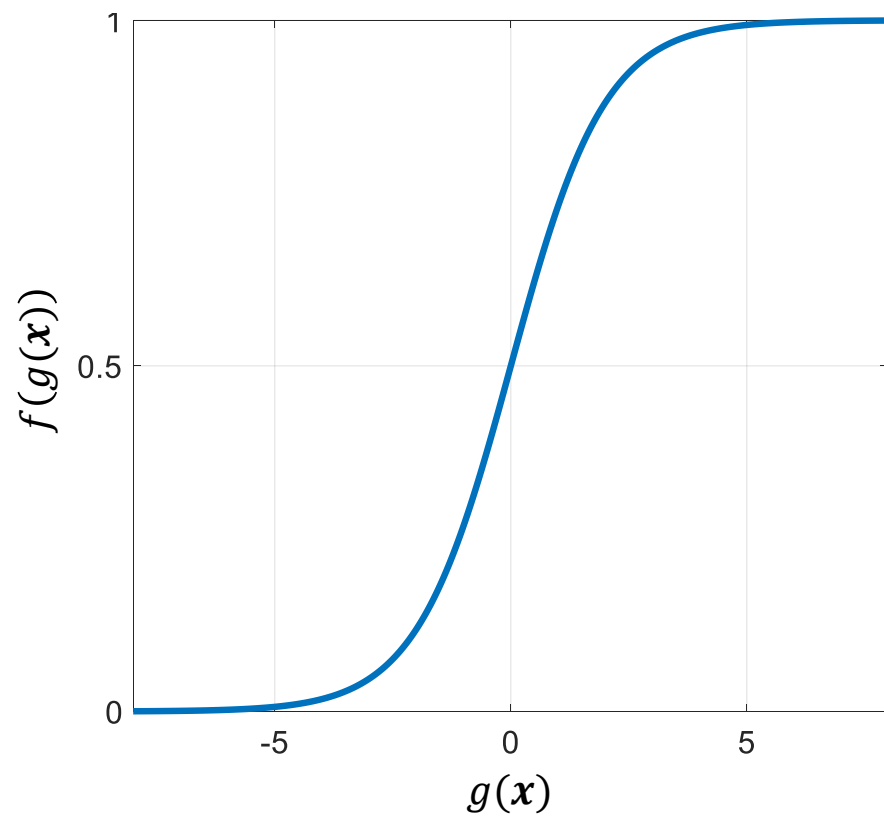
- Portanto, combinando os 2 resultados anteriores, temos que o **vetor gradiente** da **função de erro médio** é dado por

$$\frac{\partial J_e(\mathbf{a})}{\partial \mathbf{a}} = -\frac{1}{N} \sum_{i=0}^{N-1} [y(i) - h_a(\mathbf{x}(i))] \mathbf{x}(i) = -\frac{1}{N} \mathbf{X}^T (\mathbf{y} - \hat{\mathbf{y}}).$$

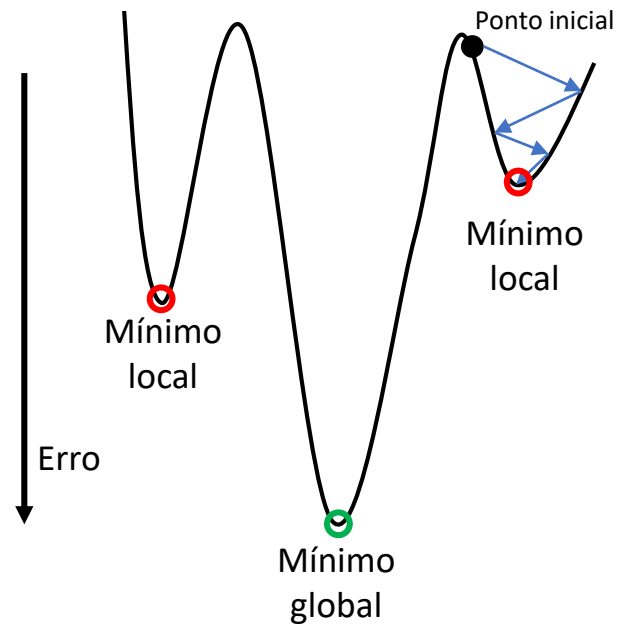
Forma matricial:
 $\mathbf{X} \in \mathbb{R}^{N \times K+1}$, $\mathbf{y}$
e $\hat{\mathbf{y}} \in \mathbb{R}^{N \times 1}$



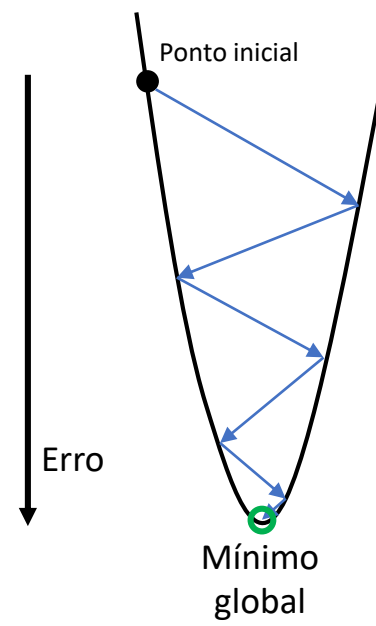
Figuras



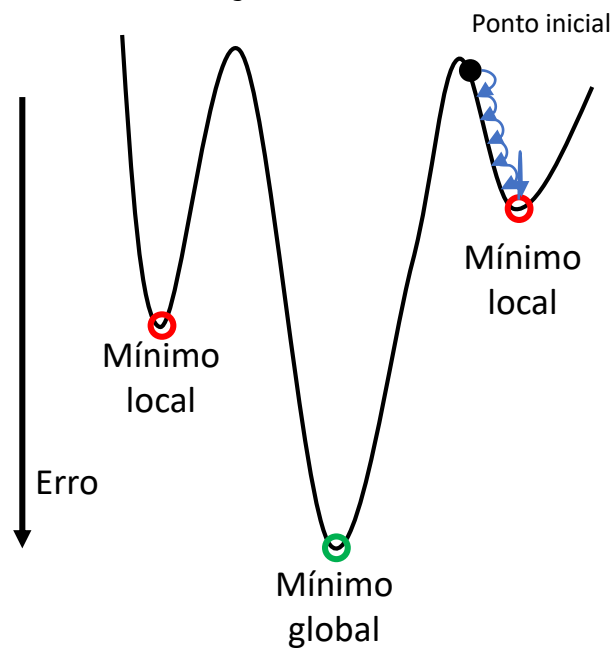
Função não-convexa



Função convexa



Função não-convexa



Função convexa

